Scalable Runtime Data Architectures for Low-Latency On-Device Speed-Limit Inference

Raphael Volz
Pforzheim University
raphael.volz@hs-pforzheim.de

March 6, 2026

Abstract

This paper evaluates runtime data architectures for low-latency, offline-capable smartphone speed-limit inference using country-scale OpenStreetMap (OSM) data. We identify four data processing scenarios that are compared to find an optimal query and update scenario along three typical query conditions. Hence, we identify the preferred operating point for practical deployment for on-device speed-limit inference as required for Intelligent speed assistance (ISA) applications that are mandatory for new vehicles in the EU since 2022 and open the way for a smartphone and open data based retrofitting approach for vehicles that are already in traffic. Thereby we can strongly support the EU traffic safety vision of reduced crash severity.

Keywords: Open Data OpenStreetMap map matching mobile GIS offline-first spatial indexing file-based preprocessing

1 Introduction

Excessive speed remains a major factor in crash severity, making reliable speed-limit assistance a core road-safety problem. For intelligent speed assistance (ISA), this creates a practical requirement for map context retrieval that remains accurate and low-latency under storage, memory, and connectivity constraints.

European regulation has made ISA deployment operationally mandatory. Article 6 of Regulation (EU) 2019/2144 requires ISA in new motor vehicles of categories M and N [1], and Delegated Regulation (EU) 2021/1958 specifies detailed ISA performance and test rules [2]. The rollout is phased: the requirement applies to new vehicle types from July 2022 and to all newly registered cars, vans, trucks, and buses from July 2024 [3]. The European Commission estimates that the broader General Safety Regulation package can prevent approximately 25,000 deaths and at least 140,000 serious injuries in the EU by 2038 [3].

A large legacy-vehicle gap remains. Eurostat reports that EU passenger-car stock exceeded 260 million vehicles in 2024 [4], while ACEA reports 10.6 million new EU car registrations in 2024 [5]. Using registration year as a coarse proxy, even an optimistic post-2024 assumption still leaves roughly 249 million pre-2024 vehicles (about 96% of the fleet) as potential retrofit targets.

This gap motivates a broad deployment approach based on smartphone sensing and open geospatial data. Eurostat reports mobile-phone internet use of 89% in cities and 82% in rural areas in 2024 [6], and OpenStreetMap (OSM) offers large-scale, continuously updated road-network attributes relevant for speed-limit inference [7; 8].

Against this background, this study examines runtime data architecture as the primary systems bottleneck. We compare four scenarios built from the same national OSM extract: global country-scale indexes (S1), tiled file packs (S2), a single-file spatially indexed database (S3), and a spatially indexed database with additional tile-window prefiltering (S4). The objective is to quantify latency and update-operability trade-offs under both cloud-like and on-device conditions.

2 Related Work

Map matching has been studied extensively for GPS trajectories under measurement noise and sampling sparsity. A foundational survey by Quddus et al. [9] systematized geometric, topological, and probabilistic matching strategies and outlined scaling challenges that remain relevant for mobile deployment.

Probabilistic sequence models became dominant for noisy and sparse data. Newson and Krumm [10] introduced a Hidden Markov Model (HMM) formulation that combines emission and transition probabilities, while Lou et al. [11] proposed ST-matching to jointly model spatial and temporal consistency for low-sampling trajectories. Real-time variants for streaming probe data were then explored in online HMM frameworks [12; 13].

Subsequent work strengthened route-level inference in sparse regimes. Rahmani and Koutsopoulos [14] formulated path inference from sparse floating-car data on urban networks, and the Path Inference Filter by Hunter et al. [15] provided a model-based low-latency approach for probe vehicles. Across these HMM/sequence families, the common effect is to keep trajectories on the current road unless turn transitions are sufficiently plausible under topology and motion evidence [10; 15; 12; 13]. Hybrid designs combining shortest-path priors with trajectory evidence further improved low-frequency matching [16], and route-choice-aware HMM approaches improved robustness for noisy sparse traces [17]. Enhanced HMM models that explicitly incorporate topology and traffic constraints also reported improved accuracy and consistency on floating-car data [18].

This structure is mirrored in production engines. OSRM, Valhalla (Meili), and Mapbox map matching expose candidate emission controls, transition plausibility settings (including tunable GPS uncertainty or search radii), and confidence-style outputs for downstream consumers [19; 20; 21]. To reduce online transition-check cost, precomputed transition structures have been proposed: FMM uses a UBODT-style table that turns many route-distance transition checks into fast lookups during inference [22].

These studies primarily optimize candidate selection and path inference given an available road graph. In contrast, our contribution focuses on the systems layer around map data itself: physically tiled, content-addressed assets with independent update delivery and bounded local IO at query time. This targets a complementary bottleneck that is critical for country-scale, offline-first mobile speed-limit inference.

Spatial access methods are central to this systems layer. The R-tree introduced by Guttman [23] established the foundational dynamic index for multi-dimensional range search and remains a standard basis for geographic candidate retrieval in database systems. Here, R-tree-based indexing is evaluated not as a new algorithmic contribution, but as an architectural instrument to control candidate fan-out and improve practical query latency at country scale.

3 Technical Vision

3.1 Purpose

YouSpeed exists to make intelligent speed assistance practical for vehicles already on the road, not only for newly manufactured cars. The product target is an offline-first smartphone assistant that infers local speed limits with low latency, high availability, and transparent data provenance. The project target is broader: to show, with reproducible evidence, that open geospatial data combined with robust runtime architecture can provide safety-relevant functionality at consumer scale.

This work is intentionally dual-track. One track is scientific and publication-oriented, where architecture decisions are benchmarked and documented under controlled conditions. The second track is product-oriented, where validated architecture choices are integrated into an iPhone app under real update, reliability, and operational constraints.

3.2 Strategic phases

The system evolution is explicitly phased:

1. **Open-data bootstrap:** initialize the platform from OpenStreetMap and rule-aware defaults for country-scale baseline coverage.
2. **Product stabilization:** prioritize fast offline inference, deterministic updates, and measurable reliability on consumer devices.
3. **Community enrichment:** add user-observed speed-sign signals once install base and observation density are sufficient.

Crowdsourcing is defined as an augmentation layer, not a replacement for OSM baseline data.

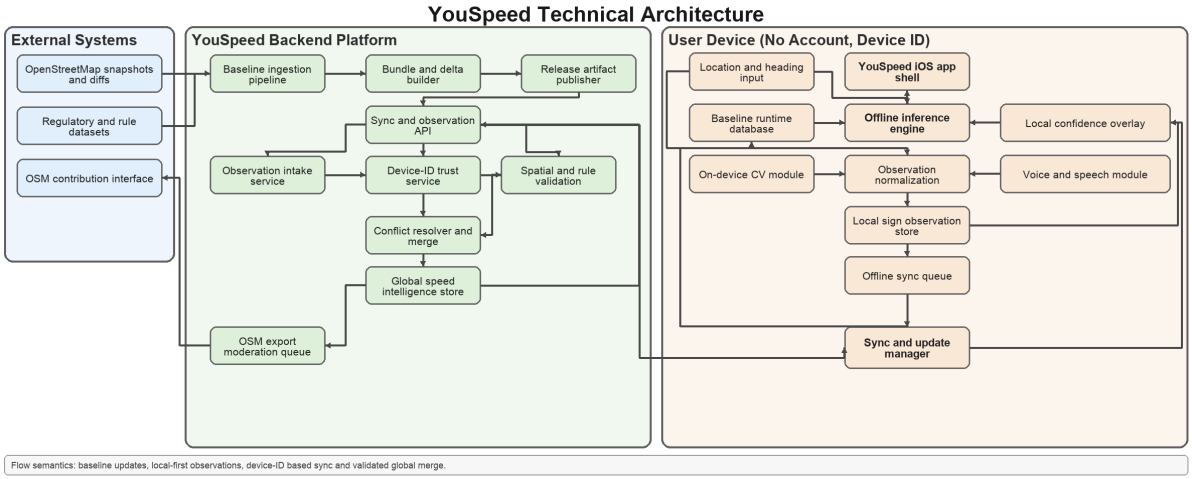

Figure 1: Technical architecture overview for baseline ingestion, on-device inference, local observation capture, and global merge/publish loops.

3.3 Crowdsourced speed intelligence

With sufficient adoption, two user-facing observation channels are introduced while driving. The first is on-device computer vision, where the smartphone camera detects speed-relevant signs when road-forward view is available. The second is voice input with speech recognition for hands-free reporting of sign changes, temporary restrictions, or map inconsistencies. Both channels generate candidate observations rather than immediate shared truth.

Each observation first updates a local device layer with confidence and decay semantics. Observations are then queued for deferred synchronization and promotion only after cross-device corroboration and validation.

3.4 Trust model without user accounts

YouSpeed is designed to operate without account authentication. Identity is pseudonymous and device-based. This lowers user friction and supports privacy goals, but shifts quality assurance into architecture: per-device source scoring, temporal/spatial consistency checks, minimum independent confirmations, and explicit conflict resolution.

A single-device report must not overwrite shared truth. Promotion to global data requires corroboration and consistency with geometry and legal plausibility constraints.

3.5 Layered data ownership and feedback loops

The data model is layered rather than monolithic:

- **External baseline layer:** imported from OSM and related official/open sources.
- **Internal global cache layer:** confirmed community observations maintained by YouSpeed.
- **Local device layer:** unconfirmed and recently confirmed observations relevant to one user.
- **Publication layer:** quality-gated candidates eligible for upstream contribution.

Confirmed findings follow two outbound paths: immediate use in the internal global cache and optional moderated feedback to OpenStreetMap via controlled export.

3.6 Synchronization as a core systems challenge

The central systems challenge is synchronization across truth sources with different latency and trust levels: local observations, globally confirmed internal data, external map updates, and periodic app bundles. Required capabilities include deterministic merge rules, versioned records, conflict precedence, rollback-safe activation, bandwidth-aware delta sync, offline accumulation, delayed upload, and eventual convergence without destructive overwrite.

3.7 App-side integration challenges

Computer vision and speech recognition are treated as separate subsystems with independent quality gates, failure modes, and runtime budgets. Integration happens through a shared observation contract so both channels emit comparable events for local storage, synchronization, and trust evaluation.

3.8 Regional sharding for large countries

To control runtime artifact size at country scale, bundle generation follows a threshold rule: if a country PBF exceeds 1 GB, the pipeline switches from a monolithic country bundle to regional Geofabrik extracts (for example German Bundeslaender). Each regional v3 bundle carries explicit coverage metadata consisting of a fast bounding box and a region polygon (`.poly`) artifact.

On device, multiple downloaded regional bundles can coexist. Runtime bundle routing is location-driven: the app first applies bbox prefiltering and then performs polygon containment for the current GPS fix to select the best local DB. This keeps inference fully offline while avoiding mandatory download of a full-country database when regional subsets are sufficient.

3.9 North-star outcome

The intended end state is a trusted open-data ISA retrofit platform that remains offline-capable and low-latency while improving continuously through external baselines and validated community observations. In that state, day-to-day user utility, auditable data operations, and reproducible research claims are aligned.

3.10 Implemented runtime inference pipeline (current app)

To avoid concept drift between architecture text and implementation, this subsection documents the currently implemented per-fix inference order in the iPhone runtime:

1. **Candidate retrieval (RTree prefilter).** For each GPS fix, the app queries `ways_rtree` in a bounded radius and loads way attributes from `ways`; if available, `way_geom` polyline points are used for exact geometry distance.
2. **Geometric and directional scoring.** Candidate score is distance plus heading mismatch penalty. Heading contributes only when vehicle speed is at least 8 km/h and heading accuracy is better than 45°; mismatch is weighted by $\lambda_h = 1.8\,\mathrm{m/deg}$.

3. **Continuity context and hysteresis.** The matcher keeps a short continuity context consisting of the previous matched way, recent matched ways, recent street refs, recent tunnel candidates, and a recent GPS-signal-loss flag. If a previous matched way remains plausible, the runtime may keep it within a small slack (12 m) and an accuracy-adjusted distance bound, reducing oscillation between nearby candidates.
4. **Same-ref transition handling.** If the current best candidate shares the same normalized `ref` as the previous way, the matcher may promote a segment transition instead of holding the old segment. This transition is preferred once the old segment is near an endpoint and the alternative segment is not materially farther away. The goal is to stabilize progression along one numbered route while still allowing segment changes at intersections.
5. **Speed-source derivation on the selected way.** Speed precedence on the matched way is: explicit `maxspeed` parse, then inherited regulatory tags (`maxspeed:type`/`source:maxspeed`: urban/rural/motorway), then highway-class prior.
6. **Inside-city classification (primary rule).** If the matched way class is one of `residential`, `service`, `crossing`, or `living_street`, the fix is classified as in-city directly.
7. **Inside-city classification (fallback rule).** For other way classes, the runtime evaluates residential polygon containment in `areas.residential`. This polygon path is also used by the benchmarked polycontainment query.
8. **City label resolution for UI and logging.** City name/source metadata is resolved from area/place context (`areas`, `city_boundary`, `city_ring`) and attached to the result for traceability; this label path is not the primary in-city classifier.
9. **Effective speed output and fallback.** At app level, final speed precedence is local per-way override first, then selected-way speed. If selected speed comes only from coarse highway class and in-city status is known, legal urban/rural default (50 km/h/100 km/h) overrides the class prior. If no way speed exists but a way match and in-city status are available, legal default is applied. If only in-city status is available without a way speed, conservative 50 km/h fallback is used.
10. **Tunnel context gating.** Tunnel candidates are filtered before final selection. A tunnel-tagged way is accepted only if it is the previous matched way, if it is connected through shared route identity (`ref`) to recent continuity context, or after a recent GPS-signal loss when the tunnel candidate was already observed in the nearby-candidate set. This reduces false switches to unrelated tunnel-tagged geometry.
11. **Tunnel context output.** If the selected way passes tunnel gating and carries truthy `tunnel`, the fix is marked tunnel segment. The UI switches to tunnel icon emphasis, keeps the matched speed limit visible, and suppresses penalty notices while tunnel mode is active.
12. **Context output for downstream logic.** Each lookup also returns `way_id`, street display name (`street_name`/`ref`), city label/source, candidate counts, timing, and tunnel flag.

3.11 Tunnel mode state machine

Tunnel handling in the current app is deterministic but no longer purely tag-driven:

- Session state is `inactive` or `active`.
- On each fix, `isTunnelSegment` from the gated matched-way decision drives the state (`active` if true, else `inactive`).
- Entry into tunnel mode is therefore possible only through continuity to the previous route context or after a recent signal-loss event that still has a remembered tunnel candidate.
- If lookup service is unavailable, continuity context records a recent GPS-signal-loss event and tunnel state is reset until the next successful gated match.
- There is still no dedicated portal detector, no inertial tunnel tracking, and no explicit exit hysteresis beyond the next successful non-tunnel match.

3.12 Runtime bundle structures used by inference

The implemented inference stack depends on attributes that are now materialized in scenario bundles (S1–S4), including:

- **Way-level context in `ways`:** `street_name`, `ref`, `approx_heading_deg`, `service`, `tunnel`, `highway`, plus speed-related tags.
- **Area context in `areas`:** `boundary`, `admin_level`, `residential`, `parking`, and geometry/bbox data for containment and city-label resolution.
- **Coverage metadata in manifests:** `coverage.bbox` and optional `coverage.poly` artifacts for regional bundle routing on device.
- **Bundle update artifacts:** region-scoped manifests, optional split DB parts, rolling delta-index manifests, and compressed SQL patches for incremental bundle refresh on device.

4 International Fine Rules and Country Bundles

4.1 Motivation

The architecture now treats speed-limit inference and sanction inference as two coupled but separable concerns. Speed-limit inference is map-driven and query-latency critical. Fine/warning inference is regulation-driven and jurisdiction-specific. For cross-border operation, country-specific legal rules must be versioned and deployed with the corresponding map bundle, otherwise the app can combine correct speed-limit lookup with incorrect warning semantics.

This extension targets the 10 countries with the highest maxspeed availability (Netherlands, Romania, Luxembourg, Belgium, Sweden, Liechtenstein, Iceland, Germany, Monaco, United Kingdom), while keeping the same runtime interface for future country additions.

4.2 Country-bundle approach

In the current repository state, generated v3 manifests carry region-scoped artifact names and explicit region identity:

- `region`: active bundle region identifier (for example `germany`, `karlsruhe-regbez`).
- `country_code`: ISO alpha-3 identifier for the legal regime of the bundle.
- `db`/`db_parts`: main runtime database artifact(s), currently named `<region>_speeds.sqlite` (or split `.partNNN` assets).
- `self`: manifest artifact, currently named `<region>_manifest.json`.

For large countries that are region-sharded, all regional manifests share the same `country_code`. Rule selection is resolved at runtime from this code; the app loads bundled `<ISO3>-rules.json` and falls back to DEU rules if country-specific rules are unavailable. The manifest field `penalty_rules` is supported by the app parser as an optional override artifact, but the currently generated manifests use bundled rule files as the default.

This makes legal-rule deployment deterministic, cacheable, and auditable in the same way as map artifacts, while preserving offline operation.

4.3 Common fine-rule data model

To support heterogeneous national enforcement schemes with one runtime parser, the fine model is normalized into a shared JSON schema with optional fields. Table 1 summarizes the core contract.

Table 1: Common traffic-fine rule model used across country bundles.

Field	Type	Semantics
`format`, `schema_version`	string, int	Rule-file format/version guard for backward-compatible parsing.
`country_code`, `country_name`	string	Jurisdiction identity used by app/UI and manifest linkage.
`currency_code`	string	Display currency for fine amounts (for example EUR, GBP, SEK).
`source_url`, `source_checked_at`	string	Provenance and verification timestamp for legal tables.
`bands[]`	array	Overspeed bands, each with `min_delta_kmh`/`max_delta_kmh`.
`severity`	enum	`money_only` or `points_and_fine`.
`money_fine_eur`	int?	Legacy numeric fine field consumed by app logic; interpreted together with `currency_code` (name kept for backward compatibility).
`penalty_points`	int?	Driving-register points when applicable.
`driving_ban_months`	int?	Immediate ban duration when explicitly defined.
`conditional_driving_ban_months`, `driving_ban_condition`	int?, string?	Conditional sanctions (for example repeat-offense, court discretion).
`locality_variants`	object?	Optional in-city/out-city split (`innerorts`/`ausserorts` or urban/rural aliases).

The model is intentionally permissive: countries with point systems, court-based escalation, or purely monetary penalties can all be represented without changing app-side decoding logic.

4.4 Cross-country sanction comparison (top 10)

To make cross-country differences explicit for deployment planning, Table 2 compares indicative sanctions across the 10 countries with the highest maxspeed availability. Overspeed bands are duplicated into in-city and rural rows while keeping a single country-column set.

Table 2: Indicative speeding-sanction matrix for the 10 countries with highest maxspeed availability (country columns; rows grouped by in-city and rural context, each sorted by excess speed). Cell format is `fine/ban-months`; currency is fixed per country header, and * marks conditional driving bans. Many countries currently provide one national band set without explicit locality split, so in-city and rural entries are identical there.

Excess (km/h)	NLD (EUR)	ROU (RON)	LUX (EUR)	BEL (EUR)	SWE (SEK)	LIE (CHF)	ISL (ISK)	DEU (EUR)	MCO (EUR)	GBR (GBP)
In-city										
1–10	50/0	405/0	49/0	53/0	2000/0	120/0	20000/0	30/0	68/0	100/0
11–20	120/0	607/0	145/0	93/0	2800/0	250/0	30000/0	50–70/0	68/0	100/0
21–30	240/0	810/0	145/0	173/0	3200/0	600/0	45000/0	115–180/0–1*	135/0	100/0
31–40	350/0	1215/0	145/0	253/0	4000/0	900/0	65000/0	260/1	375/1*	150/0
41+	480+/2*	1822+/3*	250+/1–3*	400+/3*	5000+/2*	1200+/3*	90000+/2*	400+/1–3	750+/2–3*	1000+/1*
Rural										
1–10	50/0	405/0	49/0	53/0	2000/0	120/0	20000/0	20/0	68/0	100/0
11–20	120/0	607/0	145/0	93/0	2800/0	250/0	30000/0	40–60/0	68/0	100/0
21–30	240/0	810/0	145/0	173/0	3200/0	600/0	45000/0	100–150/0–1*	135/0	100/0
31–40	350/0	1215/0	145/0	253/0	4000/0	900/0	65000/0	200/0–1*	375/1*	150/0
41+	480+/2*	1822+/3*	250+/1–3*	400+/3*	5000+/2*	1200+/3*	90000+/2*	320+/1–3	750+/2–3*	1000+/1*

4.5 Operational semantics and limitations

The app uses the rules for *driver warning and context* only. It does not claim legal finality. Rule files are treated as versioned indicative policy tables that require periodic legal verification, especially

where sanctions depend on offender history, discretionary court decisions, or local procedural details not observable in map data.

From a systems perspective, this design keeps the query/update trade-off unchanged for geometric inference while adding a low-cost legal-context layer that scales linearly with country coverage.

5 Methodology

5.1 Speed regulations in Germany

For passenger-car inference in Germany, this study uses the legal baseline defined in §3 StVO: 50 km/h in built-up areas and 100 km/h outside built-up areas unless traffic signs specify otherwise [24]. At runtime, final speed precedence is: local user override (if present), explicit way-level `maxspeed`, inherited regulatory tags, highway-class prior, and finally legal urban/rural fallback. Built-up-area inference follows the implemented app order: highway-class precedence (`residential`/`service`/`crossing`/`living_street` ⇒ in-city), then residential polygon containment for all other classes. DE built-up-area sign markers (`traffic_sign=DE:310/311`) are retained as optional metadata but are not part of the primary classifier due sparse coverage.

To frame practical impact, Germany also applies escalating sanctions for speeding. Public guidance aligned with the current fine catalog reports, for example, +21 to +25 km/h in urban areas as EUR 115 plus one point in the driving register, and from +31 km/h as EUR 260 plus one point and a one-month driving ban [25]. These enforcement consequences motivate conservative fallback design in ISA research.

5.2 OpenStreetMap dataset potential for speed-limit inference

Tables 3 and 4 summarize the Germany OSM input used throughout this study. The dataset contains 7.96 million car-drivable ways and broad coverage of context objects (administrative boundaries, area contexts, and city place tags), which is sufficient to support both direct speed-limit lookup and fallback context inference.

The data also shows strong promise for practical speed-limit inference. More than 2.63 million drivable ways carry explicit `maxspeed` tagging, while high-volume road classes such as residential, secondary, and tertiary roads contribute substantial additional network coverage for rule-based inference when explicit values are absent. This combination of explicit speed tags and structured road semantics is a key enabler for scalable, offline-capable ISA research on open data.

Beyond classic speed tags, current runtime bundles carry the tunnel and urban-context features required by the implemented app logic: way-level `tunnel` attributes for tunnel-mode signaling, and area context (`residential`, `boundary`, `admin_level`, `parking`) for city/rural fallback and city labeling.

The `maxspeed` counts in Table 4 refer to the base OSM key `maxspeed=*` on road ways (not derived keys such as `maxspeed:type`); they are distinct from area-context objects used for built-up-area inference.

Table 5 separates explicit and implicit speed-limit sources under German rule-aware inference. Beyond static snapshot downloads, OSM also supports incremental maintenance through replication diffs. In this study, daily diffs are used to quantify update feasibility and update workload for country-scale operation.

5.3 Data processing scenarios (S1–S4)

5.3.1 Shared preprocessing and scope

All four scenarios are generated from the same OSM extract (Geofabrik `germany-latest.osm.pbf`) [26]. Preprocessing follows a file-based parsing workflow [27] and does not rely on a separately deployed database server. The retained road network includes car-drivable `highway=*` categories (major roads, local roads, and connector links), ensuring that scenario differences arise from runtime data architecture and query execution rather than from differing road subsets.

Table 3: Way counts by `highway` tag, with OSM-wiki interpretation and `maxspeed` tag occurrences [7; 8].

`highway` tag	Human-readable interpretation (OSM wiki)	`maxspeed` tagged (#)	Ways (#)
`service`	Access roads to premises, parking, and service areas.	144,112	3,928,743
`residential`	Roads primarily serving housing areas.	1,183,710	2,011,658
`secondary`	Secondary inter-settlement connectors in the network hierarchy.	425,993	506,931
`tertiary`	Tertiary connectors between local centers.	351,790	477,476
`unclassified`	Minor public through roads (not "unknown class").	152,237	417,457
`primary`	Primary roads with high network importance below trunk/motorway.	215,400	240,674
`living_street`	Traffic-calmed shared street (Wohnstraße / verkehrsberuhigter Bereich).	13,543	173,199
`motorway`	Controlled-access motorway (Autobahn-type) corridor.	69,075	70,200
`motorway_link`	Motorway ramps/connectors.	17,736	43,213
`trunk`	Major non-motorway long-distance routes.	31,594	33,284
`trunk_link`	Connector ramps between trunk and other classes.	10,486	21,538
`primary_link`	Connector ramps related to primary roads.	8,861	17,140
`secondary_link`	Connector ramps related to secondary roads.	8,262	13,854
`tertiary_link`	Connector ramps related to tertiary roads.	4,207	7,839
`road`	Temporary placeholder until precise class is surveyed.	14	275
Total	**All evaluated drivable `highway` categories in this study.**	**2,637,020**	**7,963,481**

Table 4: Germany dataset profile for evaluation.

Metric	Value
Input file size (`germany-latest.osm.pbf`)	4.70 GB
Car-drivable ways in evaluated classes	7,963,481
Ways with `maxspeed=*` tag	2,637,020
Ways with `zone:maxspeed=*` tag	172,116
Ways with explicit speed-limit tag (`maxspeed` or `zone:maxspeed`)	2,639,693
Administrative city-boundary candidates (`boundary=administrative`, `admin_level` 8/9)	44,621
Total area-context objects	129,459
Place objects tagged `city`	84

Table 5: Speed-limit provenance in the Germany dataset.

Category	Ways (#)	Share (%)
Explicit tagged limit (`maxspeed` or `zone:maxspeed`)	2,639,693	33.15
Implicit regulatory tag (`maxspeed:type`/`source:maxspeed` = DE urban/rural)	78,147	0.98
Default-rule fallback (context-based legal default)	5,245,641	65.87

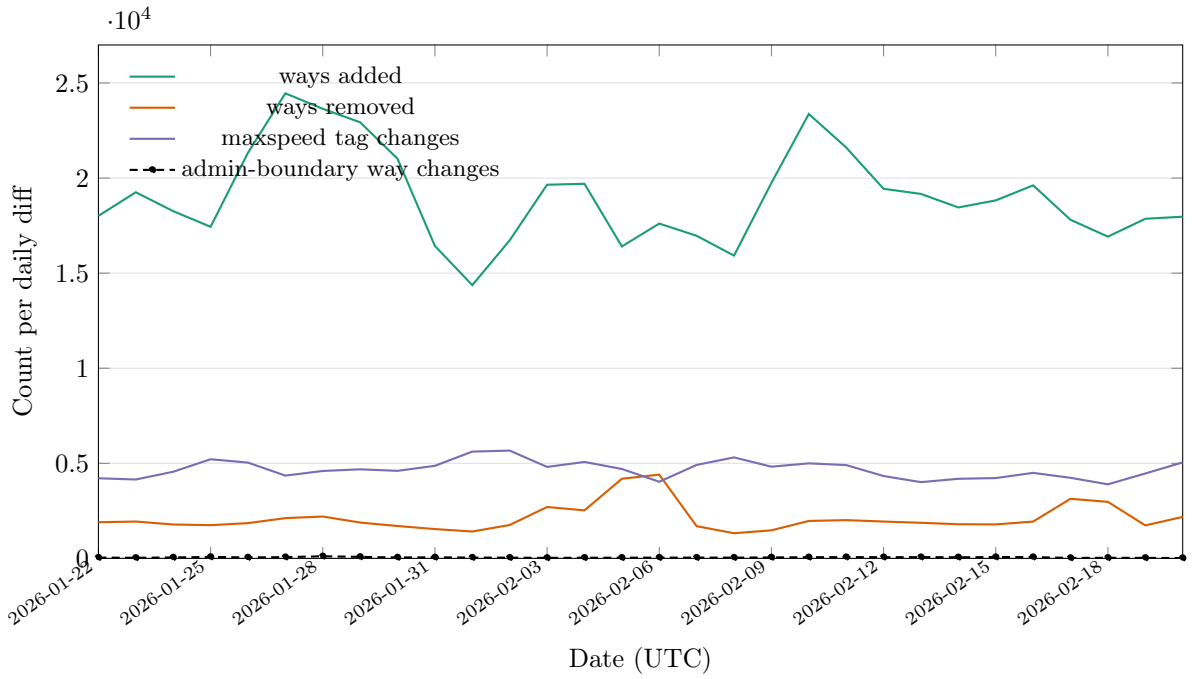

Figure 2: Daily update dynamics in the 30-day window: ways added, ways removed, `maxspeed`-tag changes, and administrative-boundary way changes per daily diff.

5.4 Three query types

We use typical location-based queries to lookup speed limits for a vehicle in motion. All queries use the same score:

$$\text{score} = d + \lambda_h \cdot \Delta\theta + p_u$$

where d is the geometry term, λ_h heading weight (m/deg), $\Delta\theta$ bidirectional heading mismatch, and p_u optional unknown-highway penalty.

bbox query. d is haversine distance from the probe to the clamped point on a candidate bbox. It is the fastest approximation and is used as the coarse filter in every scenario.

polyline query. d is minimum point-to-segment distance across the full candidate geometry. This gives the highest geometric fidelity, especially on curved and elongated ways.

hybrid query. Candidates are first ranked with bbox distance. A limited top-N subset is then refined with polyline distance, combining near-polyline quality with lower compute cost.

5.4.1 Data processing architectures (S1-S4)

- S1 is the global-index baseline: country-scale index files are queried directly without prior spatial partitioning, which makes it simple but increases broad candidate scans and IO pressure for each lookup.
- S2 introduces explicit spatial tiling and content-addressed tile assets. This localizes reads to relevant map partitions and supports selective data refresh at tile granularity.
- S3 uses a file-based embedded database with spatial indexing for candidate retrieval. S3 performs direct spatial-window search and then geometric ranking.
- S4 also uses a file-based embedded database but adds a tile-membership prefilter before spatial-index intersection to reduce candidate fan-out in dense urban regions.

Because S2–S4 runtime artifacts are file-based and immutable per build, they support horizontal cloud

Table 6: Compared runtime scenarios (same input data, different runtime packaging/query path).

Scenario	Runtime artifact	Query characteristics
S1	Global country-scale index files	Broad candidate scans with comparatively high IO per query.
S2	Content-addressed spatial tile packs	Localized reads and independently refreshable regional partitions.
S3	Single-file spatially indexed embedded database	Direct spatial-index candidate retrieval with bounded fan-out.
S4	Spatially indexed embedded database with tile-window prefiltering	Two-stage filtering (tile prefilter then spatial-index intersection) for dense-area pruning.

scaling through replication across stateless workers and object storage distribution, while also remaining suitable for autonomous offline execution on mobile devices.

5.5 Measurement scenarios

We evaluate two deployment modes:

- **Online (cloud-like) scenario:** host-side execution on a desktop-class environment, representing server-style continuous evaluation for online queries (measured without networking overhead).
- **Offline (on-device) scenario:** physical smartphone execution with country-scale runtime data stored in the mobile sandbox, representing autonomous offline operation.

Both modes are assessed the same way across the four runtime scenarios and the same three query types.

6 Evaluation

6.1 Experimental setup

The evaluated baseline was generated on 2026-02-23 from `germany-latest.osm.pbf`. Evaluation used a dual-platform setup: host-side cloud-like execution on a MacBook Pro (Mac16,5; Apple M4 Max, 16 cores; 48 GB RAM; macOS 15.5, Xcode 16.4) and offline execution on a physical iPhone 14 Pro (iPhone15,2; arm64e/A16 generation; iOS 26.2.1).

The benchmark protocol uses a fixed Berlin probe point $(52.5200, 13.4050)$, heading 90°, top-$k = 5$, and four measured query components (`bbox`, `hybrid`, `polyline`, `polycontainment`).

Runtime footprint is strongly architecture-dependent: S1 uses 7 runtime files (5.51 GB), S2 uses 109,333 files (5.10 GB), S3 uses a single file (2.60 GB), and S4 uses a single file (3.44 GB). A representative country-scale on-device run completed successfully on iPhone with one executed test and 48.331 s action elapsed time.

6.2 Benchmark results

Table 7 reports the joint online (cloud-like) and offline (device) latency matrix. Figure 3 orders the same data by scenario (S1–S4), each shown as adjacent online/offline columns.

To convert on-device latency into a raw location-update budget under worst-case per-fix operation (both way query and polygon containment query), this study uses:

$$f_{\text{cycle}} = \frac{1000}{t_{\text{maxspeed}} + t_{\text{polycontainment}}}, \qquad v_{\max} = 3.6 \cdot f_{\text{cycle}} \cdot \Delta s$$

Table 7: Joint online/offline benchmark matrix at the benchmark coordinate (ms).

Execution	Mode	S1 avg (ms)	S2 avg (ms)	S3 avg (ms)	S4 avg (ms)
online (cloud)	bbox	5451.24	172.21	64.74	166.26
online (cloud)	hybrid	10152.12	169.25	29.56	64.00
online (cloud)	polyline	10300.65	170.65	38.88	72.03
online (cloud)	polycontainment	1.51	1.07	0.94	0.44
offline (device)	bbox	4526.75	232.58	43.77	792.13
offline (device)	hybrid	2935.52	61.03	24.10	99.57
offline (device)	polyline	3122.36	76.20	39.16	78.54
offline (device)	polycontainment	3.14	0.81	0.74	2.79

where f_{cycle} is per-fix cycle throughput in Hz, t_{maxspeed} is way-query latency, $t_{\text{polycontainment}}$ is polygon containment latency, and Δs is distance between location fixes (here $\Delta s = 10\,\text{m}$). Table 8 reports per-scenario device throughput and the corresponding theoretical speed envelope without additional pipeline overheads.[1]

Table 8: On-device raw query throughput and theoretical speed envelope ($\Delta s = 10\,\text{m}$, worst-case per-fix latency $t_{\text{maxspeed}} + t_{\text{polycontainment}}$, no additional processing overhead).

Scenario	Mean latency (ms)	Mean throughput (Hz)	Best-mode $v_{\max}$ (km/h)	Worst-mode $v_{\max}$ (km/h)
S1	3531.35	0.28	12.2	7.9
S2	124.08	8.06	582.1	154.2
S3	36.42	27.46	1449.3	808.8
S4	326.20	3.07	442.7	45.3

Compared to S1, host-side speedups range from 31.7× (S2 bbox) to 343.4× (S3 hybrid). On device, the same overall ordering remains for hybrid and polyline ($S3 < S2 < S4 \ll S1$), while bbox in S4 regresses versus S3.

Based on current results, **S3** is selected as default on-device runtime: it delivers the lowest hybrid/polyline latency while keeping a single-file footprint smaller than S4.

For polygon containment (in-city/out-city classification), the measured city-context retrieval stage is 3.14 ms (S1), 0.81 ms (S2), 0.74 ms (S3), and 2.79 ms (S4). At the Berlin benchmark probe, all scenarios resolve *Mitte* as *inside-city* via explicit administrative polygon containment. The low millisecond-scale polycontainment latencies are expected because RTree prefiltering reduces candidate polygons to a very small set before exact ring containment checks.

7 Incremental Data Updates

Incremental update behavior was evaluated on a 30-day daily-diff window ending immediately before the baseline snapshot date (2026-01-22 to 2026-02-20). The analysis quantifies daily network churn, speed-tag churn, and scenario-specific refresh effort under a common observation period for the Germany subset of OSM.

Across the measured window, S4 patching is 1.33× slower than S3 in mean daily patch runtime for maxspeed updates and 4.30× slower for polygon updates, while S2 invalidates substantially fewer partitions than S1. This indicates that tile partitioning improves refresh locality over global-index invalidation, and that S3 remains the stronger update/query compromise among database-centered variants.

To translate these architectural differences into deployment-relevant transfer times, payload transmission

[1] From a full-system perspective, location-fix cadence rather than query runtime is the dominant upper bound. Android compatibility requirements specify that handheld devices must report GPS location and GPS clock measurements at least once per second (≥1 Hz) [28]. The Android location framework also exposes millisecond-level request and minimum-update intervals [29], so higher cadences can be requested, but delivered rates remain device- and policy-dependent. With $\Delta s = 10\,\text{m}$, a guaranteed 1 Hz fix stream corresponds to 36 km/h; a requested 5 Hz stream corresponds to 180 km/h.

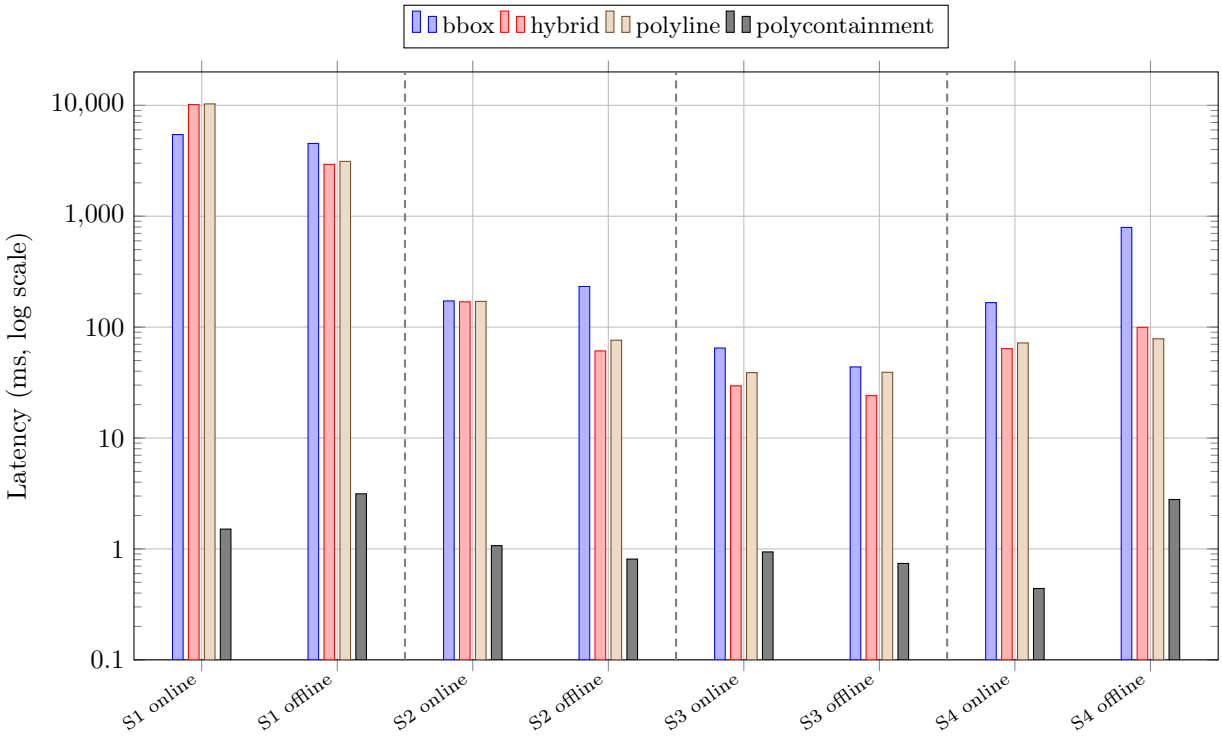

Figure 3: Scenario-first latency view derived from Table 7, including `polycontainment`. For each scenario (S1–S4), online and offline columns are adjacent, and dashed vertical lines delimit scenario groups.

Table 9: Daily-diff update workload summary (2026-01-22 to 2026-02-20, 30 days).

Metric	Total	Mean / day
Changed ways (+, −, modified)	1,829,171	60,972
`maxspeed`-related tag changes	139,999	4,667
Administrative-boundary way changes (`boundary=administrative`, level 8/9)	1,493	49.8
Invalidated S1 grid cells (S1 proxy)	201,355	6,712
Invalidated S2 tiles (S2 proxy)	80,536	2,685
S3 simulated patch runtime (S3)	30 719.38 ms	1023.98 ms
S4 simulated patch runtime (S4)	40 710.02 ms	1357.00 ms
Polygon-update S1 invalidated grid cells	15,151	505.03
Polygon-update S2 invalidated tiles	3,507	116.9
Polygon-update S3 simulated patch runtime	87.38 ms	2.91 ms
Polygon-update S4 simulated patch runtime	375.82 ms	12.53 ms

is represented as $T = (B_{\text{payload}} + B_{\text{overhead}})/R$, with B_{overhead} modeled as a separate 10% transport overhead term. For Europe, a practical throughput range is 104.3 Mbps to 217.6 Mbps for 5G downloads (spectrum-dependent) [30], with an EU-27 mobile average of 124 Mbps in 2025 [31]. Using measured median tile-pack size (47 199 B) and measured daily update volumes, indicative transfer times are: (i) 100 median tiles (4.72 MB payload): 0.19 s to 0.40 s; (ii) 1000 median tiles (47.2 MB): 1.91 s to 3.98 s; (iii) S2 mean daily invalidation volume (2685 median tiles, 126.73 MB): 5.13 s to 10.69 s; and (iv) S3 daily SQL patch transmission (gzip estimate, 5.07 MB): 0.21 s to 0.43 s. These results indicate that, in practical mobile operation, compact SQL patch delivery is typically faster than broad tile-bundle refresh for equivalent daily update coverage.

The daily benchmark is designed to be extended into a weekly longitudinal protocol with regular baseline checkpoints and intervening incremental updates. This will quantify whether reduced update effort is preserved over longer horizons while retaining low-latency query behavior.

8 Conclusion

Under a 50%/50% decision rule (equal weight on query latency and update effort), this study selects S3 as the preferred operating point. Query performance uses on-device mean latency, while update effort uses scenario-specific incremental-update proxies from Section 5 (partition invalidation for S1/S2, simulated patch runtime for S3/S4), normalized to a common $[0, 1]$ scale before weighting. The resulting ranking is $S3 \prec S4 \prec S2 \ll S1$ (lower is better).

Table 10: Scenario suitability matrix (++: best; +: suitable).

Scenario	Query time	Update time
S1	–	–
S2	+	+
S3	++	++
S4	+	+

Reproducibility

All preprocessing, scenario construction, and benchmark steps are parameterized and versioned in the accompanying research codebase available at https://github.com/volzinnovation/youspeed.de. The evaluation reported here is tied to the Germany snapshot dated 2026-02-23, with fixed probe location, heading, query modes, and scenario definitions stated in the methodology and evaluation sections.

A European maxspeed-share ranking (including Germany)

Table 11 reports the full country list used for the Europe-wide maxspeed-tag coverage comparison. Countries are ranked by

$$\text{share} = \frac{\#(\texttt{maxspeed} - \mathit{taggeddrivableways})}{\#(\text{drivable ways})} \cdot 100$$

using the same drivable `highway=*` class set as the main evaluation.

Table 11: European country ranking by `maxspeed` share (descending), including Germany.

Rank	Country	ISO2	Drivable ways (#)	`maxspeed` tagged (#)	Share (%)
1	Netherlands	NL	1,360,101	921,426	67.75
2	Romania	RO	671,248	419,884	62.55
3	Luxembourg	LU	57,401	30,766	53.60
4	Belgium	BE	816,110	302,452	37.06
5	Sweden	SE	1,235,544	443,443	35.89
6	Liechtenstein	LI	5,407	1,836	33.96
7	Iceland	IS	61,433	20,751	33.78
8	Germany	DE	7,964,480	2,637,436	33.12
9	Monaco	MC	1,224	355	29.00
10	United Kingdom	GB	4,871,056	1,275,216	26.18
11	Andorra	AD	3,714	955	25.71
12	Switzerland	CH	862,842	218,757	25.35
13	Hungary	HU	514,402	126,714	24.63
14	Austria	AT	1,257,719	287,600	22.87
15	Czech Republic	CZ	860,607	187,689	21.81
16	France	FR	7,111,322	1,490,044	20.95
17	Serbia	RS	381,641	77,588	20.33
18	Slovakia	SK	367,681	74,103	20.15
19	Malta	MT	26,092	4,658	17.85
20	Ireland and Northern Ireland	IE	1,096,604	195,628	17.84
21	Denmark	DK	886,408	157,710	17.79
22	Finland	FI	996,944	167,987	16.85
23	Spain	ES	2,720,041	439,358	16.15
24	Belarus	BY	534,645	85,966	16.08
25	Croatia	HR	298,960	47,342	15.84
26	Norway	NO	1,207,883	189,060	15.65
27	Poland	PL	3,030,384	467,033	15.41
28	Lithuania	LT	327,952	49,525	15.10
29	Estonia	EE	182,364	26,243	14.39
30	Cyprus	CY	115,415	16,038	13.90
31	Bulgaria	BG	368,235	49,724	13.50
32	Italy	IT	4,005,392	539,504	13.47
33	Montenegro	ME	56,044	7,498	13.38
34	Macedonia	MK	68,219	8,943	13.11
35	Latvia	LV	263,163	32,967	12.53
36	Faroe Islands	FO	13,226	1,654	12.51
37	Slovenia	SI	252,487	29,961	11.87
38	Greece	GR	888,354	104,360	11.75
39	Portugal	PT	870,579	102,083	11.73
40	Albania	AL	125,492	11,242	8.96
41	Ukraine (with Crimea)	UA	1,441,203	114,036	7.91
42	Georgia	GE	189,701	11,521	6.07
43	Bosnia-Herzegovina	BA	214,701	10,729	5.00
44	Moldova	MD	188,319	9,175	4.87
45	Turkey	TR	1,902,558	59,063	3.10

References

[1] European Parliament and Council of the European Union. Regulation (eu) 2019/2144 on type-approval requirements for motor vehicles and their trailers, and systems, components and separate technical units intended for such vehicles. https://eur-lex.europa.eu/eli/reg/2019/2144/oj, 2019. Accessed: 2026-02-23.

[2] European Commission. Commission delegated regulation (eu) 2021/1958 supplementing regulation (eu) 2019/2144 by laying down detailed rules concerning intelligent speed assistance and driver drowsi-

ness and attention warning systems. https://eur-lex.europa.eu/eli/reg_del/2021/1958/oj, 2021. Accessed: 2026-02-23.

[3] European Commission. New mandatory vehicle safety features from july 2024. https://single-market-economy.ec.europa.eu/news/new-mandatory-vehicle-safety-features-2024-07-05_en, 2024. Accessed: 2026-02-23.

[4] Eurostat. Passenger cars in the eu (statistics explained; data extracted in january 2026). https://ec.europa.eu/eurostat/statistics-explained/SEPDF/cache/25886.pdf, 2026. Accessed: 2026-02-23.

[5] European Automobile Manufacturers' Association (ACEA). Car registrations: +0.8% in 2024; battery electric 13.6% market share in december. https://www.acea.auto/pc-registrations/new-car-registrations-0-8-in-2024-battery-electric-13-6-market-share-in-december/, 2025. Accessed: 2026-02-23.

[6] Eurostat. Internet access in cities and rural areas. https://ec.europa.eu/eurostat/web/products-eurostat-news/w/ddn-20241122-1, 2024. Accessed: 2026-02-23.

[7] OpenStreetMap Wiki Contributors. Key:highway. https://wiki.openstreetmap.org/wiki/Key:highway, 2026. Accessed: 2026-02-23.

[8] OpenStreetMap Wiki Contributors. Key:maxspeed. https://wiki.openstreetmap.org/wiki/Key:maxspeed, 2026. Accessed: 2026-02-23.

[9] Mohammed A. Quddus, Washington Y. Ochieng, and Robert B. Noland. Current map-matching algorithms for transport applications: State-of-the art and future research directions. *Transportation Research Part C: Emerging Technologies*, 15(5):312–328, 2007. doi: 10.1016/j.trc.2007.05.002. URL https://doi.org/10.1016/j.trc.2007.05.002.

[10] Paul Newson and John Krumm. Hidden markov map matching through noise and sparseness. In *Proceedings of the 17th ACM SIGSPATIAL International Conference on Advances in Geographic Information Systems*, pages 336–343, 2009. doi: 10.1145/1653771.1653818. URL https://doi.org/10.1145/1653771.1653818.

[11] Yin Lou, Chengyang Zhang, Yu Zheng, Xing Xie, Wei Wang, and Yan Huang. Map-matching for low-sampling-rate gps trajectories. In *Proceedings of the 17th ACM SIGSPATIAL International Conference on Advances in Geographic Information Systems*, pages 352–361, 2009. doi: 10.1145/1653771.1653820. URL https://doi.org/10.1145/1653771.1653820.

[12] C. Y. Goh, J. Dauwels, N. Mitrovic, M. T. Asif, A. Oran, and P. Jaillet. Online map-matching based on hidden markov model for real-time traffic sensing applications. In *2012 15th International IEEE Conference on Intelligent Transportation Systems*, pages 776–781, 2012. doi: 10.1109/ITSC.2012.6338627. URL https://doi.org/10.1109/ITSC.2012.6338627.

[13] Zhao-cheng He, She Xi-wei, Li-jian Zhuang, and Pei-lin Nie. On-line map-matching framework for floating car data with low sampling rate in urban road networks. *IET Intelligent Transport Systems*, 7 (4):404–414, 2013. doi: 10.1049/iet-its.2011.0226. URL https://doi.org/10.1049/iet-its.2011.0226.

[14] Mahmood Rahmani and Haris N. Koutsopoulos. Path inference from sparse floating car data for urban networks. *Transportation Research Part C: Emerging Technologies*, 30:41–54, 2013. doi: 10.1016/j.trc.2013.02.002. URL https://doi.org/10.1016/j.trc.2013.02.002.

[15] Timothy Hunter, Pieter Abbeel, and Alexandre Bayen. The path inference filter: Model-based low-latency map matching of probe vehicle data. *IEEE Transactions on Intelligent Transportation Systems*, 15(2):507–529, 2014. doi: 10.1109/TITS.2013.2282352. URL https://doi.org/10.1109/TITS.2013.2282352.

[16] Mohammed Quddus and Simon Washington. Shortest path and vehicle trajectory aided map-matching for low frequency gps data. *Transportation Research Part C: Emerging Technologies*, 55:328–339, 2015. doi: 10.1016/j.trc.2015.02.017. URL https://doi.org/10.1016/j.trc.2015.02.017.

[17] George R. Jagadeesh and Thambipillai Srikanthan. Online map-matching of noisy and sparse location data with hidden markov and route choice models. *IEEE Transactions on Intelligent Transportation Systems*, 18(9):2423–2434, 2017. doi: 10.1109/TITS.2017.2647967. URL https://doi.org/10.1109/TITS.2017.2647967.

[18] Mingliang Che, Yingli Wang, Chi Zhang, and Xinliang Cao. An enhanced hidden markov map matching model for floating car data. *Sensors*, 18(6):1758, 2018. doi: 10.3390/s18061758. URL https://doi.org/10.3390/s18061758.

[19] Project OSRM Contributors. Osrm api documentation: Match service. https://project-osrm.org/docs/v5.24.0/api/#match-service, 2026. Accessed: 2026-02-26.

[20] Valhalla Contributors. Valhalla meili documentation. https://valhalla.github.io/valhalla/meili/, 2026. Accessed: 2026-02-26.

[21] Mapbox. Map matching api. https://docs.mapbox.com/api/navigation/map-matching/, 2026. Accessed: 2026-02-26.

[22] Can Yang and Gyözö Gidófalvi. Fast map matching, an algorithm integrating hidden markov model with precomputation. *International Journal of Geographical Information Science*, 32(3):547–570, 2018. doi: 10.1080/13658816.2018.1474007. URL https://doi.org/10.1080/13658816.2018.1474007.

[23] Antonin Guttman. R-trees: A dynamic index structure for spatial searching. In *Proceedings of the 1984 ACM SIGMOD International Conference on Management of Data*, pages 47–57, 1984. doi: 10.1145/602259.602266. URL https://doi.org/10.1145/602259.602266.

[24] Bundesministerium der Justiz. Straßenverkehrs-ordnung (stvo), section 3 (geschwindigkeit). https://www.gesetze-im-internet.de/stvo_2013/__3.html, 2013. Accessed: 2026-02-23.

[25] ADAC. Bußgelder für geschwindigkeitsüberschreitungen (germany). https://www.adac.de/news/bussgeld-erhoeht-geschwindigkeitsueberschreitung/, 2025. Accessed: 2026-02-23.

[26] Geofabrik GmbH. Geofabrik openstreetmap data extracts – germany latest. https://download.geofabrik.de/europe/germany-latest.osm.pbf, 2026. Accessed: 2026-02-23.

[27] PyOsmium Contributors. Pyosmium documentation. https://docs.osmcode.org/pyosmium/latest/, 2026. Accessed: 2026-02-23.

[28] Android Open Source Project. Android 16 compatibility definition. https://source.android.com/docs/compatibility/16/android-16-cdd, 2026. Accessed: 2026-02-23.

[29] Android Open Source Project. Locationrequest api source documentation (frameworks/base). https://android.googlesource.com/platform/frameworks/base/+/refs/heads/main/location/java/android/location/LocationRequest.java, 2026. Accessed: 2026-02-23.

[30] Opensignal. Europe's mobile network experience 2024. https://www.opensignal.com/reports/2024/08/europe-mobile-network-experience, 2024. Accessed: 2026-02-23.

[31] European Commission. Impact assessment report accompanying the white paper "how to master europe's digital infrastructure needs?" (swd(2026) 13 final). https://eur-lex.europa.eu/legal-content/EN/TXT/?uri=CELEX:52026SC0013, 2026. Accessed: 2026-02-23.